Objektumorientált programozás
2. Beadandó

Dokumentáció

Ballai Péter Lóránt
EYSEED

2026.04.19

Tartalom

Feladatleírás ... 3
Fejlesztői dokumentáció ... 6
Típusleírás és Reprezentáció ... 6
Műveletek ... 7
Osztálydiagram ... 7
Implementáció ... 8
Kolónia osztály megvalósítása ... 8
A ragadozó osztály megvalósítása ... 8
A zsákmány osztály megvalósítása ... 9
Nyúl osztály megvalósítása ... 10
Lemming osztály megvalósítása ... 10
Ürge osztály megvalósítása ... 10
Hóbagoly osztály megvalósítása ... 10
Sarki róka osztály megvalósítása ... 10
Farkas osztály megvalósítása ... 10
Kolónia gyár osztály megvalósítása ... 11
Program osztály megvalósítása ... 12
Tesztelési terv ... 13
Szekvencia Diagram ... 14

Feladatleírás

Objektumelvű/Objektumorientált programozás
2. beadandó feladat
Tundra

A tundra élővilágát zsákmányállat-kolóniák és ragadozókolóniák alkotják, amelyek egyedszámai hatással vannak a többire. Ha a zsákmányállatok száma nagyon nagy, nagyrészük elvándorol, mert nem talál élelmet. Ha a ragadozók száma nagy, gyorsabban fogy a zsákmányállatok száma, mert vadásznak rájuk.

Három ragadozófajt különböztetünk most meg: hóbaglyot, sarki rókát és farkast. Háromfajta zsákmányállat van: lemming, sarki nyúl és ürge. Az egyes kolóniáknak van beceneve, faja illetve egyedszáma. A következőkben megadjuk, hogy a zsákmányállat-kolóniák mikén reagálnak a rájuk vadászó ragadozókolóniákra:

Lemming: ha vadásznak rájuk, az egyedszámuk a rájuk vadászó kolónia számának 4-szeresével csökken. Minden második körben megduplázódik az egyedszámuk. 200 fölötti egyedszámnál csökken a számuk 30-ra.

Nyúl: ha vadásznak rájuk, az egyedszámuk a rájuk vadászó kolónia számának 2-szeresével csökken. Minden második körben másfélszeresére nő az egyedszámuk. 100 fölötti egyedszámnál csökken a számuk 20-ra.

Ürge: ha vadásznak rájuk, az egyedszámuk a rájuk vadászó kolónia számának 2-szeresével csökken. Minden negyedik körben megduplázódik az egyedszámuk. 200 fölötti egyedszámnál csökken a számuk 40-re.

A ragadozó kolóniák minden körben véletlenszerűen kiválasztanak egy zsákmány kolóniát, amit megtámadnak. Ha nincs elegendő állat a megtámadott kolóniában (azaz nincs annyi egyed, amennyivel fogy a zsákmányok száma, ha megtámadják őket), akkor fogy a ragadozók száma is: közül minden negyedik elpusztul. Minden nyolcadik körben a hóbaglyok 4 egyedenként 1, a rókák 4 egyedenként 3, a farkasok 4 egyedenként 2 utódot fialnak.

Feladat: Addig szimuláljuk a folyamatot, amíg minden zsákmány kolónia ki nem hal, vagy a zsákmányállatok összesített száma meg nem négyszereződik a kiinduló értékhez képest. Körönként mutassuk meg a kolóniák összes tulajdonságát!

A program egy szövegfájlból olvassa be a kolóniák adatait! Az első sorban a zsákmánykolóniák és a ragadozó kolóniák darabszámai szerepelnek szóközzel elválasztva. A következő sorok
tartalmazzák a kolóniák adatait szóközökkel elválasztva: a becenevüket (szóközök nélküli szöveg), a fajukat (amit egy karakter azonosít: h - hóbagoly, s – sarki róka, f – farkas, l – lemming, n – nyúl, u - ürge), és a kezdeti egyedszámukat.

A program kérje be a fájl nevét, majd jelenítse is meg a tartalmát. Feltehetjük, hogy a fájl formátuma helyes. Kezelje le a szimuláció végtelenített futásának eshetőségét!

Objektumelvű/Objektumorientált programozás
2. beadandó feladat
Tundra

Egy lehetséges bemenet:
4 2
kicsik l 86
picik l 90
szaporak n 26
szorgosak u 12
ehesek f 12
tollasak h 6

Szükség szerint írjon saját segédfüggvényeket! A problémákat (pl. hibás beadott értékek)
kezelje le konzisztensen valamely tanult módszer segítségével.

2-es:
- Hozzon létre a feladat adatainak a tárolásához a megfelelő osztályszerkezetet.
- Írjon egy osztályt, ami képes az adott fájlformátumot betölteni, és az előzőleg létrehozott osztályok listájává alakítani (Factory pattern).
- Írassa ki a konzolra az összes betöltött adatot rendesen megformázva.
- Menüpontok segítségével minden funkció legyen tesztelhető a konzolon.
- Legyen minden függvénynek Unit tesztje (MSTest Test Project).

3-as:
- 2-es követelmények teljesülnek.
- Írjon fejlesztői dokumentációt az elkészített programhoz (végezhető műveletek, osztálydiagram, implementált függvények rövid leírása és ábrázolása választott módszerrel, tesztelési terv).

4-es:
- 2-es, 3-as követelmények teljesülnek.
- Egészítse ki az osztályhierarchiát a feladat megoldásához szükséges virtuális függvényekkel, és felüldefiniálásaikkal, majd ezeket használva oldja meg a szükséges feladatot.
- A konzolos tesztprogram legyen kiegészítve az új függvényekkel.
- Legyenek az új függvényekhez is Unit tesztek.

5-ös:
- 2-es, 3-as, 4-es követelmények teljesülnek.
- Egészítse ki az elkészített dokumentációt az új funkcionalitásokkal.
- Készítsen az egyik előző (nem triviális) függvényhez vagy funkcionalitáshoz szekvencia diagramot, vagy 2 tetszőleges (nem triviális) függvényhez formális definíciót (specifikációt). *

* nem triviális függvény = összetettebb feladatmegoldás

Fejlesztői dokumentáció

Típusleírás és Reprezentáció

Kolónia (Colony)

Típusleírás: Egy általános, absztrakt állatkolóniát leíró típus, amely összefogja a közös tulajdonságokat.

Reprezentáció: Name (string): A kolónia beceneve.

Species (char): A fajt azonosító karakter.

Population (int): Az aktuális egyedszám.

Zsákmány (Prey) – Absztrakt

Típusleírás: A zsákmányállatok közös viselkedését (túlszaporodás, veszteség vadászatkor) megvalósító típus.

Reprezentáció: Örökli a Colony adattagjait.

multiplier (int): Vadászatkor alkalmazott veszteségszorzó.

breedingMultiplier (double): A szaporodás mértéke.

roundsToBreed, maxPopulation, newPopulation (int): A szaporodás ciklusát és korlátait tároló változók.

Ragadozó (Predator) – Absztrakt

Típusleírás: A ragadozók közös viselkedését (vadászat, éhezés, szaporodás) megvalósító típus.

Reprezentáció: Örökli a Colony adattagjait.

neededToBreed (int): A szaporodáshoz szükséges egyedek száma.
breedAmount (int): A születő utódok száma.

Szimuláció (Program)

Típusleírás: A játék motorja, amely tárolja az állatokat és vezérli a körök lefutását.

Reprezentáció: preys (List<Prey>): A zsákmánykolóniák dinamikus listája.

predators (List<Predator>): A ragadozókolóniák dinamikus listája.

currentRound (int): A szimuláció aktuális körének számlálója.

initialPreyPopulation (int): A zsákmányok kiinduló összesített egyedszáma a leállási feltételhez.

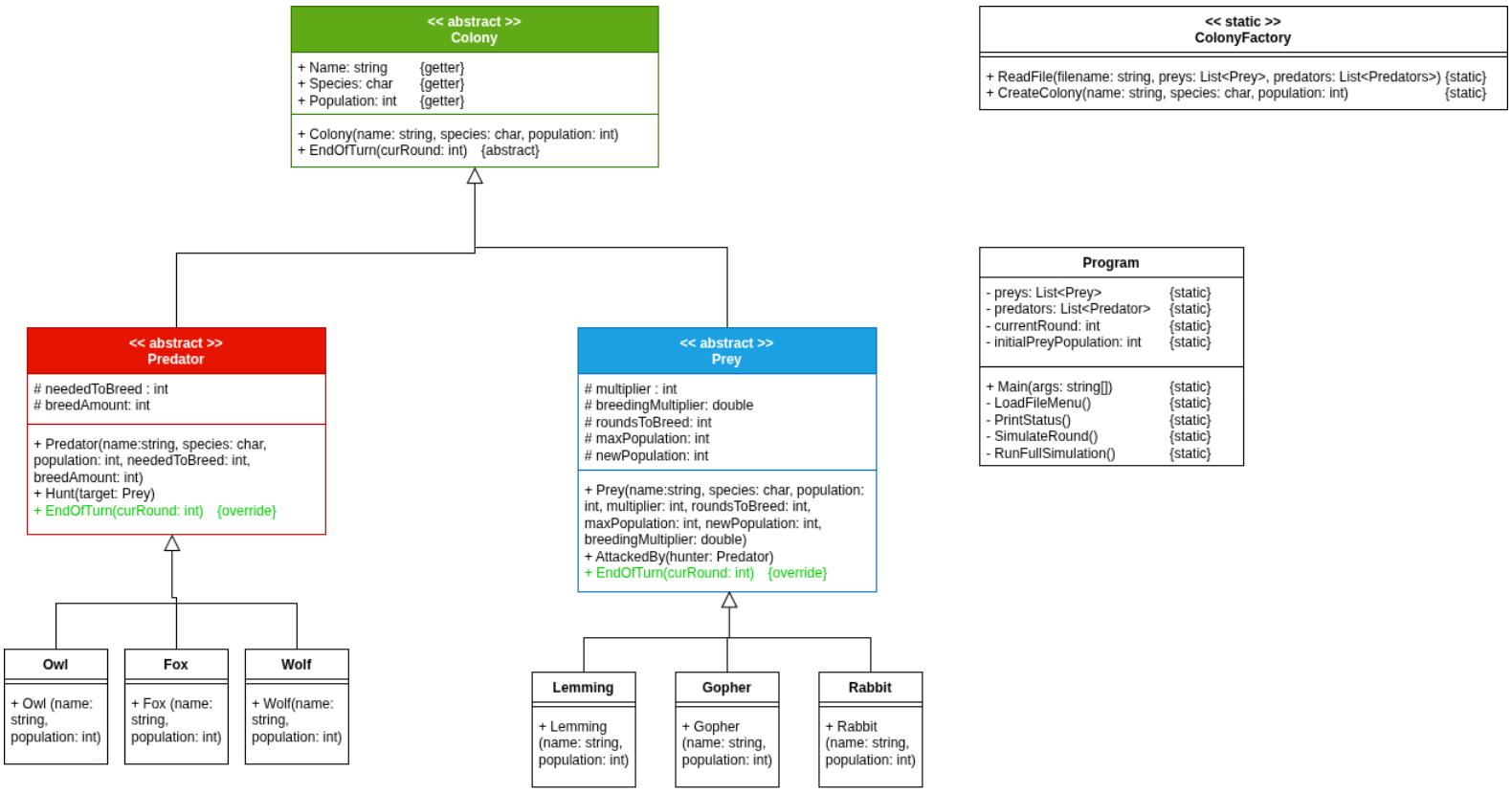
Műveletek

1. Fájl beolvasása és feldolgozása
Fájl sorainak beolvasása, osztálypéldányok tárolása.

2. Feldolgozott adatok megjelenítése
A kolóniák neve és létszáma kiírása.

3. Szimuláció futtatása
A ragadozók megpróbálják a zsákmányokat levadászni.

Osztálydiagram



Implementáció

Kolónia osztály megvalósítása

A kolónia osztály konstruktora, amit az öröklött osztályok meghívnak példányosításkor

```
Colony(name: string, species: char, population: int)
     Name := name
     Species := species
     Population := population
```

A ragadozó osztály megvalósítása

A ragadozó osztály konstruktora, amit az öröklött osztályok meghívnak példányosításkor

```
Predator(name: string, species: char, population: int,
neededToBreed: int, breedAmount: int)
     base(name, species, population)
     this.neededToBreed := neededToBreed
     this.breedAmount := breedAmount
```

A ragadozó osztály vadászat metódusa, amit az öröklött osztályok meghívnak

```
Hunt(target: Prey)
     if not target.AttackedBy(this)
          then Population := Population * 0.75
     endif
```

A ragadozó osztály kör vége metódusa, ezt a Kolónia osztályból írjuk felül, amit az öröklött osztályok meghívnak

```
EndOfTurn(curRound: int)
     if curRound % 8 != 0
          then return
     endif
     timesToBreed := Population / neededToBreed
     if timesToBreed = 0
          then return
     endif
     Population := Population + timesToBreed * breedAmount
```

A zsákmány osztály megvalósítása

A zsákmány osztály konstruktora, amit az öröklött osztályok meghívnak példányosításkor

```
Prey(name: string, species: char, population: int, multiplier: int,
roundsToBreed: int, maxPopulation: int, newPopulation: int,
breedingMultiplier: double)
     base(name, species, population)
     this.multiplier := mult  iplier
     this.roundsToBreed := roundsToBreed
     this.maxPopulation := maxPopulation
     this.newPopulation := newPopulation
     this.breedingMultiplier := breedingMultiplier
```

A zsákmány osztály megtámadott metódusa, amit az öröklött osztályok meghívnak

```
AttackedBy(hunter: Predator) : bool
     loss := hunter.Population * multiplier
     if Population >= loss
          then Population := Population –
hunter.Population*multiplier
          return true
     endif
     Population := 0
     return false
```

A zsákmány osztály kör vége metódusa, ezt a Kolónia osztályból írjuk felül, amit az öröklött osztályok meghívnak

```
EndOfTurn(curRound: int)
     if curRound % roundsToBreed != 0
          then return
     endif
     Population := breedingMultiplier * Population
     if Population > maxPopulation
          then Population := newPopulation
     endif
```

Nyúl osztály megvalósítása

A nyúl osztály konstruktora

```
Rabbit(name: string, population: int)
     base(name, ‘n’, population, 2, 2, 100, 20, 1.5)
```

Lemming osztály megvalósítása

A lemming osztály konstruktora

```
Lemming(name: string, population: int)
     base(name, ‘l’, population, 4, 2, 200, 30, 2)
```

Ürge osztály megvalósítása

Az ürge osztály konstruktora

```
Gopher(name: string, population: int)
     base(name, ‘u’, population, 2, 4, 200, 30, 2)
```

Hóbagoly osztály megvalósítása

A hóbagoly osztály konstruktora

```
Owl(name: string, population: int)
     base(name, ‘h’, population, 4, 1)
```

Sarki róka osztály megvalósítása

A sarki róka osztály konstruktora

```
Fox(name: string, population: int)
     base(name, ‘s’, population, 4, 3)
```

Farkas osztály megvalósítása

A farkas osztály konstruktora

```
Wolf(name: string, population: int)
     base(name, ‘h’, population, 4, 2)
```

Kolónia gyár osztály megvalósítása

A kolónia gyár osztály fájl beolvasó metódusa

```
ReadFile(filename: string, preys: List<Prey>, predators:
List<Predator>)
     sr := new StreamReader(filename)
     row := sr.ReadLine().Split(“ “)
     int preyCount := row[0]
     int predatorCount := row[1]
     preys.Clear()
     predators.Clear()
     totalColonies := predatorCount + preyCount
     for i=0..totalColonies loop
          row := sr.ReadLine().Split(“ “)
          name := row[0]
          species := row[1]
          pop := row[2]
          newColony := CreateColony(name, species, pop)
          if newColony is Prey p
               then preys.Add(p)
          elsif newColony is Predator pr
               then predators.Add(pr)
          endif
     endloop
```

A kolónia gyár kolónia készítő metódusa

```
CreateColony(name: string, species: char, population: int)
     switch species
          case ‘h’: return new Owl(name, population)
          case ‘s’: return new Fox(name, population)
          case ‘f’: return new Wolf(name, population)
          case ‘l’: return new Lemming(name, population)
          case ‘n’: return new Rabbit(name, population)
          case ‘u’: return new Gopher(name, population)
     endswitch
     error
```

Program osztály megvalósítása

A Program osztály kör szimuláló metódusa

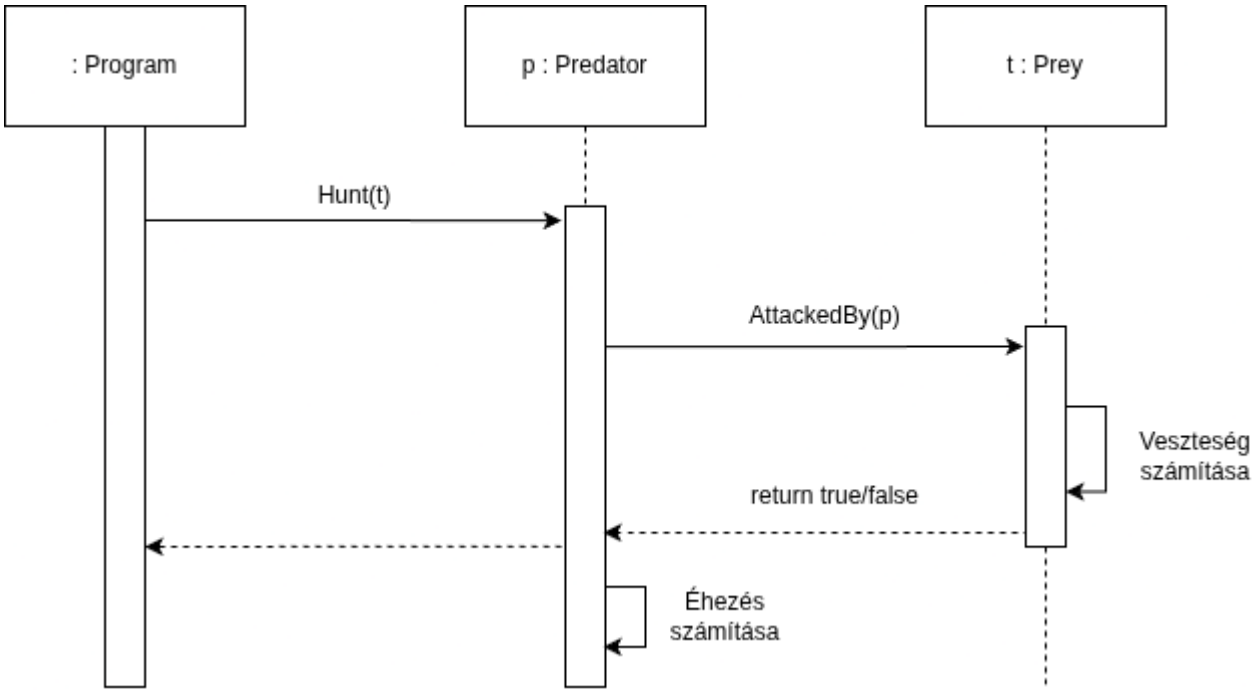
```
SimulateRound()
     rnd := new Random()
     currentRound := currentRound + 1
     if(|preys| > 0)
          then
          forall predator in predators
          loop
               target := rnd.Next(0, |preys|)
               predator.Hunt(preys[target])
          endloop
     endif
     forall prey in preys
     loop
          prey.EndOfTurn(currentRound)
     endloop
     forall predator in predators
     loop
          predator.EndOfTurn(currentRound)
     endloop
     preys.RemoveAll(p => p.Population <= 0)
     predators.RemoveAll(p => p.Population <= 0)
```

A Program egész szimuláló futtató metódusa

```
RunFullSimulation()
     preyCount := initalPreyPopulation
     maxRounds := 1000

     while preyCount > 0 and preyCount < 4*initialPreyPopulation &&
     currentRound < maxRounds
     loop
          SimulateRound()
          preyCount := 0
          forall prey in preys
          loop
               preyCount += prey.Population
          endloop
          printStatus()
     endloop
     Console.WriteLine($"\nSimulation edned after round
     #{currentRound}!")
     PrintStatus()
```

Tesztelési terv

1. Kolónia gyár (Factory) példányosításának ellenőrzése helyes faj azonosító (karakter) esetén
2. Kolónia gyár (Factory) kivételkezelésének ellenőrzése ismeretlen/hibás faj azonosító megadása esetén
3. Zsákmányállat megtámadásának (AttackedBy) ellenőrzése elegendő egyedszám (sikeres vadászat) esetén
4. Zsákmányállat megtámadásának (AttackedBy) ellenőrzése nem elegendő egyedszám (a zsákmánykolónia megsemmisülése) esetén
5. Zsákmányállat szaporodásának (EndOfTurn) ellenőrzése a fajhoz tartozó, megfelelő szaporodási körben
6. Zsákmányállat túlszaporodásának ellenőrzése (amikor a határérték elérése után lecsökken a létszám)
7. Zsákmányállat egyedszámának ellenőrzése nem szaporodási (kimaradó) körben
8. Ragadozó vadászatának (Hunt) ellenőrzése sikertelen támadás esetén (a ragadozó egyedszámának 25%-os csökkenése)
9. Ragadozó vadászatának (Hunt) ellenőrzése sikeres támadás esetén (az egyedszám stagnálása)
10. Ragadozó szaporodásának (EndOfTurn) ellenőrzése a 8. körben
11. Ragadozó egyedszámának ellenőrzése nem a 8. (hanem egy kimaradó) körben

Szekvencia Diagram